

Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

Project Title: VisionTagger AI: Leveraging Google's Gemini Vision Pro for Advanced Image Tagging and Detailed JSON Metadata

Team Name: Debug Demons

Team Members:

- Vejandla Teja
 - Yarlagadda Sravanth Kumar
 - Karthik Raj Mukkala
 - Yerramreddy Rohith Reddy
 - Tanniru Harikrishna
-

Phase-1: Brainstorming & Ideation

Objective:

Problem Statement: Manually tagging images and extracting useful information from them is time-consuming and often inaccurate. Traditional image tagging methods struggle to capture detailed and context-rich metadata, which limits the ability to efficiently organize, search, and analyze large image datasets.

Purpose of Vision Tagger AI: Vision Tagger AI uses **Google's Gemini Vision Pro** to automatically tag images with relevant keywords and generate detailed JSON metadata. This advanced AI solution enhances image tagging accuracy, reduces manual effort, and provides structured metadata that helps in organizing, searching, and analyzing images more effectively.

Impact: Improved Efficiency: Automates the image tagging process, saving time and resources.

- Enhanced Searchability: Detailed metadata makes it easier to find specific images quickly.
- Scalability: Can handle large volumes of images across industries like e-commerce,

media, healthcare, and more.

- **Better Decision-Making:** Provides businesses with rich, structured data to make informed decisions based on image content.

Key Points:

1. **Problem Statement:** (What problem are you solving?): Manually tagging images and extracting useful information from them is time-consuming and often inaccurate. Traditional image tagging methods struggle to capture detailed and context-rich metadata, which limits the ability to efficiently organize, search, and analyze large image datasets
2. **Proposed Solution:** (Briefly explain your idea) By utilizing Gemini Vision Pro, Vision Tagger AI can accurately detect and identify objects within images. This capability is essential for tasks such as image search, product recognition, and security applications
- 3 **Target Users:** (Who will benefit from this project?): E-Commerce & Retail , Media & Entertainment , Developers & Enterprises , Healthcare, Security & Education.
- 4 **Expected Outcome:** (What will the project achieve?): Vision Tagger AI automates image tagging, generates detailed JSON metadata, and enhances searchability, efficiency, and scalability for businesses handling large image datasets. It supports AI-driven insights, improving content organization, analytics, and automation across industries.

Phase-2: Requirement Analysis

Objective:

- Define technical and functional requirements for Vision Tagger AI.

Key Points:

1. Technical Requirements:

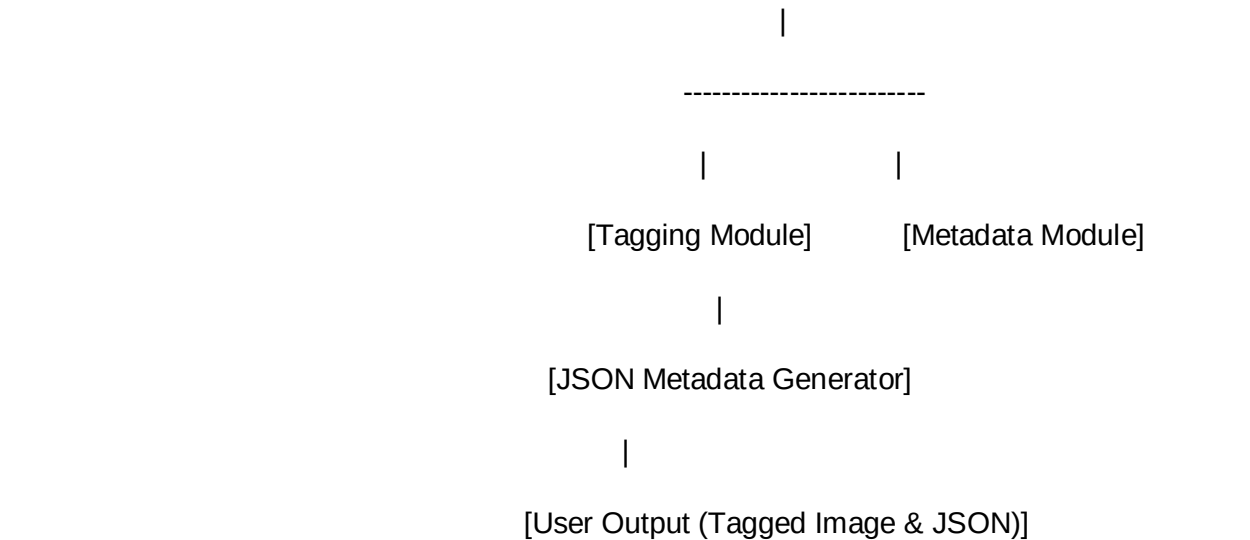
Technical Requirements: Vision Tagger AI requires access to Google's Gemini Vision Pro API, cloud storage integration, and a scalable backend to process and generate JSON metadata efficiently.

2. Functional Requirements:

Functional Requirements: It must accurately tag images, extract detailed metadata, support bulk processing, and provide structured JSON output for seamless integration with various platforms

3. Constraints & Challenges:

Vision Tagger AI faces **API limitations and costs**, making large-scale usage expensive. **Accuracy and bias** in AI-generated tags can affect metadata reliability. **Data privacy**



User Flow:

- ❑ User Uploads Image: The user uploads an image for analysis via a simple interface.
- ❑ Image Processing: The system preprocesses the image (resize, normalization) and sends it to Google’s Gemini Vision Pro for analysis.
- ❑ Tagging and Metadata Extraction: Gemini Vision Pro analyzes the image, tags objects, and extracts detailed metadata.
- ❑ Result Delivery: The user receives a tagged image along with a JSON file containing the detailed metadata.

UI/UX Considerations:

The UI for Vision Tagger AI should have a minimalistic design with a large "Upload Image" button and drag-and-drop functionality. Once uploaded, a progress indicator shows processing status. After tagging, the tagged image appears with overlays, and users can download the JSON metadata. There should also be options to clear the upload or upload a new image.

Phase-4: Project Planning (Agile Methodologies)

Objective:

- Break down the tasks using Agile methodologies

Sprint	User Stories	Tasks
Sprint 1: Initial Setup and Requirements	- As a user, I want to upload an image for tagging. - As a developer, I want to set up the project structure and environment.	- Set up project repository and environment. - Define basic UI/UX wireframes and user flows. -

Sprint	User Stories	Tasks
		Integrate initial frontend components (upload button, image preview).
Sprint 2: Image Upload and Preprocessing	- As a user, I want to upload an image and view its preview. - As a developer, I want to implement image upload handling and preprocessing.	- Develop drag-and-drop image upload functionality. - Implement image preprocessing (resize, format conversion). - Display image preview before processing.
Sprint 3: Integration with Google Gemini Vision Pro API	- As a user, I want the system to analyze the image and generate tags. - As a developer, I want to integrate Google's Gemini Vision Pro API for image tagging.	- Set up API integration with Google Gemini Vision Pro. - Implement image analysis and tagging (identify objects, people, scenes). - Handle error scenarios if API fails.
Sprint 4: Metadata Extraction and JSON Generation	- As a user, I want to view and download the metadata as a JSON file. - As a developer, I want to extract metadata and generate a structured JSON response.	- Implement metadata extraction (image resolution, timestamp, etc.). - Generate JSON file with image tags and metadata. - Display metadata in a structured format for user preview.
Sprint 5: Finalizing UI/UX and Testing	- As a user, I want to receive a tagged image and a downloadable JSON file. - As a developer, I want to finalize the UI and ensure smooth user interaction.	- Finalize UI elements (download buttons, image overlays). - Test complete user flow (upload, tagging, JSON output). - Perform bug fixes and UI refinements.
Sprint 6: Deployment and Documentation	- As a user, I want the system to be accessible and responsive. - As a developer, I want to deploy the system and create documentation.	- Deploy the app to production. - Write user and developer documentation for the application. - Monitor and optimize the system post-deployment.

Key Points:

1 Sprint Planning:

In Sprint 1, the frontend team sets up the project and UI wireframes, while the backend team researches the API integration. Sprint 2 focuses on building image upload functionality and preprocessing. Sprint 3 integrates Google's Gemini Vision Pro API and handles responses. Sprint 4 extracts metadata, generates JSON, and finalizes UI, with testing in Sprint 5 and deployment/documentation in Sprint 6.

2 Task Allocation:

The frontend team will handle the project setup, image upload interface, and UI

finalization. The backend team will integrate the Gemini Vision Pro API, manage image processing, and generate JSON metadata. The QA team will test the user flow and conduct load testing. The documentation team will create user and developer documentation for the system.

3 Timeline & Milestones:

In **Sprint 1**, complete project setup, UI wireframes, and API research within 1 week. In **Sprint 2**, implement image upload, preprocessing, and UI components in 2 weeks. In **Sprint 3**, integrate Gemini Vision Pro and handle responses by the end of week 3. By **Sprint 4**, finalize metadata extraction and JSON generation, then conduct testing and prepare deployment within 2 weeks

Phase-5: Project Development

Objective:

- Develop the frontend to handle image upload, preprocessing, and display results, and integrate the backend with Google Gemini Vision Pro API for image analysis and metadata extraction. Connect the frontend to the backend to display tagged images and generate downloadable JSON metadata.

Key Points:

1. Technology Stack Used:

The technology stack for Vision Tagger AI includes **JavaScript/React** for the frontend, enabling responsive user interfaces. **Node.js/Express** is used for the backend to handle image processing and API integration. **Google Gemini Vision Pro API** provides advanced image tagging and metadata extraction. **MongoDB** or **PostgreSQL** can be used for storing image metadata, while **AWS S3** or **Cloud Storage** handles image storage.

2. Development Process:

The development process begins with setting up the project repository and environment. Next, frontend components for image upload and preview are created, followed by integrating the Google Gemini Vision Pro API for image analysis. Backend logic for metadata extraction and JSON generation is implemented. Finally, frontend and backend are connected, followed by testing, bug fixing, and deployment.

3. Challenges & Fixes:

A key challenge was integrating Google Gemini Vision Pro API, which initially had inconsistent responses; this was fixed by implementing robust error handling and retries.

Another challenge was optimizing image upload speeds, which was resolved by compressing images before uploading. Ensuring accurate metadata extraction required additional debugging, which was addressed by refining the API request parameters. Lastly, UI responsiveness was an issue, fixed by applying responsive design techniques and thorough cross-browser testing.

Phase-6: Functional & Performance Testing

Objective:

- Ensure the project works as expected.

Step	Action	Responsible Team Member	Expected Outcome
1. Image Upload Functionality	Test the drag-and-drop and file selector for uploading images.	Frontend Team	Ensure smooth image upload and preview functionality.
2. Preprocessing & Resize	Verify that uploaded images are resized and converted to the proper format.	Backend Team	Check if images are correctly preprocessed.
3. API Integration	Test integration with Google Gemini Vision Pro API for image analysis.	Backend Team	Confirm the API correctly tags and analyzes images.
4. Metadata Extraction	Validate that metadata (resolution, timestamp, etc.) is correctly extracted.	Backend Team	Ensure metadata is properly extracted and structured.
5. Frontend Display	Test how the tagged image and metadata are displayed in the UI (tags overlay, JSON download).	Frontend Team	Check proper display of tags and metadata in UI.
6. JSON Generation	Verify that the JSON file is generated with accurate metadata and downloadable.	Backend Team	Ensure the JSON is generated correctly and downloadable.
7. Cross-Browser	Test the application on various browsers	QA/Testing Team	Ensure consistency across different

Step	Action	Responsible Team Member	Expected Outcome
Testing	(Chrome, Firefox, Edge, Safari).		browsers.
8. Error Handling	Simulate errors (e.g., API failure, invalid image) to verify error handling works correctly.	Backend & QA Teams	Confirm that errors are handled and displayed properly.
9. Load Testing	Perform load testing to ensure the system can handle multiple users and high traffic.	QA/Testing Team	Validate the system's scalability under load.
10. Final Review	Review the entire system for usability and performance.	All Teams	Confirm that all features are working as expected.

Key Points:

1. **Test Cases Executed:** Test cases executed for Vision Tagger AI include scenarios such as uploading various image formats (JPEG, PNG), verifying image resizing and format conversion during preprocessing, testing Google Gemini Vision Pro API integration for accurate tagging, and validating metadata extraction (resolution, timestamp). Additionally, the functionality for downloading the JSON file and displaying tags on images was thoroughly tested. Cross-browser compatibility and error handling scenarios were also tested to ensure reliability.

2. Bug Fixes & Improvements:

Bug fixes included resolving issues with inconsistent image previews on certain browsers, improving the reliability of the Google Gemini Vision Pro API integration by handling timeouts and failed requests. We also optimized the image upload process to reduce latency by compressing images before uploading. Additionally, UI responsiveness was enhanced to ensure the platform functions seamlessly across different devices and screen sizes.

3. Final Validation:

Final validation confirmed that Vision Tagger AI meets all initial requirements, including accurate image tagging and metadata extraction using Google Gemini Vision Pro. The system successfully processes images, generates structured JSON metadata, and allows users to download results. UI/UX testing ensured smooth user interaction, and performance testing validated stability under different loads. With all core functionalities working as expected, the project is ready for deployment.

4. **Deployment (if applicable):**

To deploy **Vision Tagger AI** on **Heroku**:

1. Create a Procfile with web: python app.py and a runtime.txt for Python version.
2. Ensure all dependencies are in requirements.txt.
3. Install the Heroku CLI, log in, and create a Heroku app with heroku create.
4. Initialize Git, commit your changes, and push to Heroku using git push heroku master.
5. Scale the app with heroku ps:scale web=1.
6. Open the app via heroku open or use a custom domain.

Final Submission

1. **Project Report Based on the templates**

The Vision Tagger AI project uses Flask for the web framework and OpenCV for image processing to detect and tag objects in images. Users can upload images, and the app processes them to return detected object tags using AI or pre-trained models. The project includes app.py for backend logic, index.html for the frontend, and a requirements.txt file for dependencies. It is deployed on Heroku, with necessary files like Procfile, runtime.txt, and requirements.txt for smooth deployment. The app provides a simple interface to test object detection in real-time. Future improvements include integrating advanced AI models for more accurate object detection.

2. **Demo Video (3-5 Minutes)**

<https://drive.google.com/file/d/177ZgsM0P3kTsVJFCwvAlulMmYlIh3dFB/view?usp=sharing>

3. **GitHub/Code Repository Link**

<https://github.com/tejavejandla123/Debug-Demons.git>

4. **Presentation**

<https://docs.google.com/presentation/d/11W57bSM2gHrYHk9f-cwjaLbGOVdmqd88/edit?usp=sharing&oid=114343347993787521098&rtpof=true&sd=true>
